

UNIT 3: BANKER’S ALGORITHM SOLVED NUMERICAL EX-AMPLES

Course Pattern: TE IT - 2024 Pattern (SPPU)
Document Type: Solved Numerical Guide

Academic Term: Semester V (2026)
Course Code: PCC-301-ITT

Problem 1: Banker’s Safety Algorithm & Safe Sequence

INTUITION:

Problem Logic & Steps:

- 1. Calculate Need Matrix:** $Need[i][j] = Max[i][j] - Allocation[i][j]$.
- 2. Initialize Vector:** Set $Work = Available$ and $Finish[i] = FALSE$ for all processes.
- 3. Iterative Search:** Find an unfinished process P_i where $Need[i] \leq Work$.
- 4. Simulate Execution:** Assume P_i completes, releases its allocated resources: $Work = Work + Allocation[i]$, set $Finish[i] = TRUE$.
- 5. Result Verification:** If all processes complete ($Finish = TRUE$), the system is in a **Safe State**.

Problem Statement

Consider a system with 5 processes (P_0, P_1, P_2, P_3, P_4) and 3 resource types (A, B, C). Total physical instances available: $A = 10, B = 5, C = 7$.

At time t_0 , the snapshot of the system is as follows:

Process	Allocation (A B C)	Max (A B C)	Available (A B C)
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Questions:

- (a) Compute the Need Matrix.
- (b) Is the system currently in a Safe State? Determine the Safe Sequence.

Step-by-Step Solution

1.2.1 Step 1: Calculation of Need Matrix (Need = Max – Allocation)

- $Need[P_0] = (7, 5, 3) - (0, 1, 0) = (7, 4, 3)$
- $Need[P_1] = (3, 2, 2) - (2, 0, 0) = (1, 2, 2)$
- $Need[P_2] = (9, 0, 2) - (3, 0, 2) = (6, 0, 0)$
- $Need[P_3] = (2, 2, 2) - (2, 1, 1) = (0, 1, 1)$
- $Need[P_4] = (4, 3, 3) - (0, 0, 2) = (4, 3, 1)$

1.2.2 Step 2: Execution of Safety Algorithm

Initialize Work = (3, 3, 2) and Finish = [F, F, F, F, F].

1. **Check P_0 :** Need[P_0] = (7, 4, 3) $\leq$ Work(3, 3, 2)? **False** (7 > 3). P_0 must wait.
2. **Check P_1 :** Need[P_1] = (1, 2, 2) $\leq$ Work(3, 3, 2)? **True!**
 - P_1 executes and finishes.
 - New Work = Work + Allocation[P_1] = (3, 3, 2) + (2, 0, 0) = (5, 3, 2).
 - Finish[P_1] = TRUE. Sequence: $\angle P_1 r \angle$.
3. **Check P_3 :** Need[P_3] = (0, 1, 1) $\leq$ Work(5, 3, 2)? **True!**
 - P_3 executes and finishes.
 - New Work = (5, 3, 2) + (2, 1, 1) = (7, 4, 3).
 - Finish[P_3] = TRUE. Sequence: $\angle P_1, P_3 r \angle$.
4. **Check P_4 :** Need[P_4] = (4, 3, 1) $\leq$ Work(7, 4, 3)? **True!**
 - P_4 executes and finishes.
 - New Work = (7, 4, 3) + (0, 0, 2) = (7, 4, 5).
 - Finish[P_4] = TRUE. Sequence: $\angle P_1, P_3, P_4 r \angle$.
5. **Check P_0 :** Need[P_0] = (7, 4, 3) $\leq$ Work(7, 4, 5)? **True!**
 - P_0 executes and finishes.
 - New Work = (7, 4, 5) + (0, 1, 0) = (7, 5, 5).
 - Finish[P_0] = TRUE. Sequence: $\angle P_1, P_3, P_4, P_0 r \angle$.
6. **Check P_2 :** Need[P_2] = (6, 0, 0) $\leq$ Work(7, 5, 5)? **True!**
 - P_2 executes and finishes.
 - New Work = (7, 5, 5) + (3, 0, 2) = (10, 5, 7).
 - Finish[P_2] = TRUE. Sequence: $\angle P_1, P_3, P_4, P_0, P_2 r \angle$.

1.2.3 Summary Conclusion

All processes finished execution (Finish[i] = TRUE for all i).

- **System State: Safe State.**
- **Valid Safe Sequence:** $\angle P_1, P_3, P_4, P_0, P_2 r \angle$.

Problem 2: Resource-Request Evaluation**Problem Statement**

Using the same system snapshot from Problem 1 (Available = (3, 3, 2)), suppose process P_1 submits a new resource request: Request[P_1] = (1, 0, 2).

Can this request be granted immediately by the OS?

Step-by-Step Solution**2.2.1 Step 1: Check Bounds**

1. Request[P_1](1, 0, 2) $\leq$ Need[P_1](1, 2, 2)? **True!**
2. Request[P_1](1, 0, 2) $\leq$ Available (3, 3, 2)? **True!**

2.2.2 Step 2: Pretend Resource Allocation to P_1

- Available = Available – Request[P_1] = (3, 3, 2) – (1, 0, 2) = (2, 3, 0).
- Allocation[P_1] = Allocation[P_1] + Request[P_1] = (2, 0, 0) + (1, 0, 2) = (3, 0, 2).
- Need[P_1] = Need[P_1] – Request[P_1] = (1, 2, 2) – (1, 0, 2) = (0, 2, 0).

2.2.3 Step 3: Run Safety Algorithm on New System State

Initialize Work = (2, 3, 0).

1. **Check P_1 :** Need[P_1](0, 2, 0) $\leq$ Work(2, 3, 0)? **True!**

- New Work = $(2, 3, 0) + (3, 0, 2) = (5, 3, 2)$.
- 2. **Check P_3 :** Need[P_3](0, 1, 1) ≤ Work(5, 3, 2)? **True!**
 - New Work = $(5, 3, 2) + (2, 1, 1) = (7, 4, 3)$.
- 3. **Check P_4 :** Need[P_4](4, 3, 1) ≤ Work(7, 4, 3)? **True!**
 - New Work = $(7, 4, 3) + (0, 0, 2) = (7, 4, 5)$.
- 4. **Check P_0 :** Need[P_0](7, 4, 3) ≤ Work(7, 4, 5)? **True!**
 - New Work = $(7, 4, 5) + (0, 1, 0) = (7, 5, 5)$.
- 5. **Check P_2 :** Need[P_2](6, 0, 0) ≤ Work(7, 5, 5)? **True!**
 - New Work = $(7, 5, 5) + (3, 0, 2) = (10, 5, 7)$.

2.2.4 Summary Conclusion

System remains in a Safe State with Safe Sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$.

- **Decision:** The OS grants the request immediately.

End of Operating Systems Unit 3 Solved Examples — SPPU TE IT 2024 Pattern